

Passo a passo - Criar containers e subir para o ECR no Linux

Projeto SysTrainTrack | Imagens: lambda-etl-s3-trigger e lambda-etl-24h

Este guia usa terminal Linux/bash. Os comandos consideram que voce esta dentro da pasta **scripts-python-systrain**. Se estiver dentro da pasta raiz **SysTrainTrack**, veja a secao de caminhos alternativos no final.

Nao cole credenciais da AWS em chats ou repositorios. Em AWS Academy/Vocareum, as credenciais temporarias geralmente ficam em AWS Details > Show AWS CLI.

1. Verificar Docker e AWS CLI

Confira se Docker e AWS CLI estao instalados e se o daemon do Docker esta rodando.

```
docker --version
aws --version
sudo systemctl status docker
```

Se o Docker nao estiver instalado no Ubuntu/Debian, use:

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable --now docker
```

Se quiser rodar Docker sem sudo, adicione seu usuario ao grupo docker e abra um novo login/sessao depois:

```
sudo usermod -aG docker $USER
newgrp docker
```

Se preferir nao mexer em grupo de usuario, use **sudo docker** nos comandos Docker. Nesse caso, use sudo tambem no docker login/tag/push para manter o mesmo contexto.

2. Configurar credenciais temporarias da AWS

No terminal Linux, exporte as credenciais da sua conta/lab. Substitua os valores abaixo pelos dados reais do AWS Academy/Vocareum.

```
export AWS_ACCESS_KEY_ID="SUA_ACCESS_KEY"
export AWS_SECRET_ACCESS_KEY="SUA_SECRET_KEY"
export AWS_SESSION_TOKEN="SEU_SESSION_TOKEN"
export AWS_DEFAULT_REGION="us-east-1"
```

Teste se a AWS reconheceu sua identidade:

```
aws sts get-caller-identity
```

Se retornar Account, Arn e UserId, suas credenciais estao funcionando.

3. Preparar variaveis usadas nos comandos

```
REGION="us-east-1"
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

echo $ACCOUNT_ID
echo $REGION
```

O ACCOUNT_ID sera usado para montar a URI do ECR, como:

ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/repositorio:tag.

4. Logar o Docker no ECR

Esse comando autentica o Docker no registry ECR da sua conta AWS.

```
aws ecr get-login-password --region $REGION | \
docker login --username AWS --password-stdin \
"${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com"
```

5. Criar e subir a imagem lambda-etl-s3-trigger

5.1 Build da imagem local

Estando dentro de **scripts-python-systain**, rode:

```
docker build -t lambda-etl-s3-trigger \
  ./pythonMainScripts/extract_transform_load/lambda-etl-s3-trigger/
```

Confira se a imagem foi criada:

```
docker images
```

Voce deve ver algo parecido com **lambda-etl-s3-trigger:latest**.

5.2 Criar o repositório no ECR

```
REPO="lambda-etl-s3-trigger"
aws ecr create-repository --repository-name $REPO --region $REGION
```

Se aparecer `RepositoryAlreadyExistsException`, o repositório já existe. Pode continuar.

5.3 Taguear a imagem local para o ECR

```
docker tag lambda-etl-s3-trigger:latest \
  "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/${REPO}:latest"
```

O docker tag não envia nada ainda; ele só cria uma referência local com o endereço do ECR.

5.4 Enviar a imagem para o ECR

```
docker push "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/${REPO}:latest"
```

URI final da imagem:

```
echo "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/lambda-etl-s3-trigger:latest"
```

6. Criar e subir a imagem lambda-etl-24h

6.1 Build da imagem local

```
docker build -t lambda-etl-24h \
  ./pythonMainScripts/extract_transform_load/lambda-etl-24h/
```

6.2 Criar o repositório no ECR

```
REPO="lambda-etl-24h"
aws ecr create-repository --repository-name $REPO --region $REGION
```

6.3 Taguear a imagem local para o ECR

```
docker tag lambda-etl-24h:latest \
  "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/${REPO}:latest"
```

6.4 Enviar a imagem para o ECR

```
docker push "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/${REPO}:latest"
```

URI final da imagem:

```
echo "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/lambda-etl-24h:latest"
```

7. Versao direta - todos os comandos

Use esta secao quando ja souber que Docker, AWS CLI e credenciais estao funcionando.

7.1 lambda-etl-s3-trigger

```
REGION="us-east-1"
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
REPO="lambda-etl-s3-trigger"

docker build -t lambda-etl-s3-trigger \
  ./pythonMainScripts/extract_transform_load/lambda-etl-s3-trigger/

aws ecr create-repository --repository-name $REPO --region $REGION

aws ecr get-login-password --region $REGION | \
  docker login --username AWS --password-stdin \
  "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com"

docker tag lambda-etl-s3-trigger:latest \
  "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/${REPO}:latest"

docker push "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/${REPO}:latest"
```

7.2 lambda-etl-24h

```
REGION="us-east-1"
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
REPO="lambda-etl-24h"

docker build -t lambda-etl-24h \
  ./pythonMainScripts/extract_transform_load/lambda-etl-24h/

aws ecr create-repository --repository-name $REPO --region $REGION

aws ecr get-login-password --region $REGION | \
  docker login --username AWS --password-stdin \
  "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com"

docker tag lambda-etl-24h:latest \
  "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/${REPO}:latest"

docker push "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/${REPO}:latest"
```

8. Se voce estiver na pasta SysTrainTrack

Os comandos de build mudam porque o caminho passa a incluir **scripts-python-systrain**.

```
docker build -t lambda-etl-s3-trigger \
  ./scripts-python-systrain/pythonMainScripts/extract_transform_load/lambda-etl-s3-trigger/

docker build -t lambda-etl-24h \
  ./scripts-python-systrain/pythonMainScripts/extract_transform_load/lambda-etl-24h/
```

Os comandos de create-repository, tag e push continuam iguais.

9. Erros comuns e como resolver

Erro	Causa provavel	Resolucao
NoCredentials	Credenciais da AWS nao foram exportadas no terminal.	Exporte AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN e AWS_DEFAULT_REGION.
Login Succeeded, mas push falha	Repositorio ECR nao existe ou tag local nao foi criada.	Rode aws ecr create-repository e depois docker tag antes do docker push.
tag does not exist	Voce mudou o REPO, mas nao rodou docker tag para esse novo repo.	Rode docker tag IMAGEM_LOCAL:latest URI_DO_ECR:latest e tente o push de novo.

invalid reference format	Alguma variavel esta vazia, geralmente REPO.	Confira com echo \$ACCOUNT_ID, echo \$REGION, echo \$REPO.
Cannot connect to Docker daemon	Docker nao esta rodando ou usuario sem permissao.	sudo systemctl start docker; ou use sudo docker; ou adicione usuario ao grupo docker.

Resumo mental: build cria a imagem local; create-repository cria o lugar no ECR; tag aponta a imagem para o endereco do ECR; push envia a imagem para a AWS.